JavaScript Data Types

What is a Data Type?

A **Data Type** tells JavaScript what kind of value a variable stores.

```
let name = "Rokon"; // String
let age = 23; // Number
```

Types of Data

JavaScript has **2 main types** of data:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

1. Primitive Data Types

Primitive data types store a **single value** and are **immutable**.

There are **7 primitive data types**:

Data Type	Description	Example
String	Text	`"Hello"`
Number	Integer or Decimal	`10` , `3.14`
Boolean	`true` or `false`	`true`
Undefined	Variable declared but not assigned	`undefined`
Null	Intentionally empty value	`null`
BigInt	Very large integer	`123456789n`
Symbol	Unique identifier	`Symbol("id")`

Examples

```
let name = "Rokon";
let age = 23;
let isStudent = true;
let city;
let car = null;
let bigNumber = 123456789012345678901n;
let id = Symbol("userId");
```

2. Non-Primitive (Reference) Data Types

Non-Primitive data types can store **multiple values** and are stored **by reference**.

Object

```
const student = {
  name: "Rokon",
  age: 23,
};

console.log(student.name);
```

Array

```
const fruits = ["Apple", "Mango", "Orange"];

console.log(fruits[0]);
```

Function

```
function greet() {
  console.log("Hello!");
}

greet();
```

Primitive vs Non-Primitive

Feature	Primitive	Non-Primitive
Stores	Single Value	Multiple Values
Mutable	No	Yes
Copied By	Value	Reference
Examples	String, Number, Boolean	Object, Array, Function

Primitive Example

```
let a = 10;
let b = a;

b = 20;

console.log(a); // 10
console.log(b); // 20
```

Explanation: `b` gets a copy of `a`, so changing `b` does not affect `a`.

Non-Primitive Example

```
let person1 = {
  name: "Rokon",
};

let person2 = person1;

person2.name = "Rahim";

console.log(person1.name); // Rahim
console.log(person2.name); // Rahim
```

Explanation: Both variables point to the same object, so changing one also changes the other.

Easy Way to Remember

- **Primitive** = Each variable has its **own copy**.
- **Non-Primitive** = Multiple variables can point to the **same object**.

Interview Question

Q: What is the difference between Primitive and Non-Primitive Data Types?

Answer:
Primitive data types store a single value and are copied **by value**, while Non-Primitive data types store multiple values and are copied **by reference**.

JavaScript Operators

What is an Operator?

An **Operator** is a symbol that performs an operation on one or more values (operands).

```
let sum = 10 + 5;

console.log(sum); // 15
```

Types of Operators

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Increment & Decrement Operators
6. String Operator
7. Ternary Operator
8. Type Operator

1. Arithmetic Operators

Used for mathematical calculations.

Operator	Meaning	Example
`+`	Addition	`10 + 5`
`-`	Subtraction	`10 - 5`
`*`	Multiplication	`10 * 5`
`/`	Division	`10 / 5`
`%`	Modulus	`10 % 3`
`**`	Exponent	`2 ** 3`

```
let a = 10;
let b = 3;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a / b);
console.log(a % b);
console.log(a ** b);
```

2. Assignment Operators

Used to assign or update values.

Operator	Example	Same As
`=`	`x = 5`	Assign value
`+=`	`x += 2`	`x = x + 2`
`-=`	`x -= 2`	`x = x - 2`
`*=`	`x *= 2`	`x = x * 2`
`/=`	`x /= 2`	`x = x / 2`
`%=`	`x %= 2`	`x = x % 2`

```
let x = 10;

x += 5;
console.log(x); // 15

x -= 3;
console.log(x); // 12
```

3. Comparison Operators

Used to compare two values.

Operator	Meaning	Example
`==`	Equal (value only)	`5 == "5"`
`===`	Strict Equal	`5 === "5"`
`!=`	Not Equal	`5 != 4`
`!==`	Strict Not Equal	`5 !== "5"`
`>`	Greater Than	`10 > 5`
`<`	Less Than	`5 < 10`
`≥`	Greater Than or Equal	`5 ≥ 5`
`≤`	Less Than or Equal	`5 ≤ 10`

```
console.log(10 > 5);
console.log(10 == "10");
console.log(10 === "10");
```

4. Logical Operators

Used to combine conditions.

Operator	Meaning
`&&`	AND
`	
`!`	NOT

```
let age = 20;
let hasID = true;

console.log(age ≥ 18 && hasID);
console.log(age < 18 || hasID);
console.log(!hasID);
```

5. Increment & Decrement Operators

Increase or decrease a value by **1**.

Operator	Meaning
`++`	Increment
`--`	Decrement

```
let count = 5;

count++;
console.log(count); // 6

count--;
console.log(count); // 5
```

6. String Operator

The `+` operator joins strings.

```
let firstName = "MD";
let lastName = "Rokonuzzaman";

console.log(firstName + " " + lastName);
```

Output:

```
MD Rokonuzzaman
```

7. Ternary Operator

A short way to write an `if ... else`.

Syntax

```
condition ? trueValue : falseValue;
```

```
let age = 20;

let result = age ≥ 18 ? "Adult" : "Minor";

console.log(result);
```

8. Type Operator

The `typeof` operator returns the data type of a value.

```
console.log(typeof "Hello");
console.log(typeof 100);
console.log(typeof true);
console.log(typeof []);
console.log(typeof {});
```

Operator Precedence

Operators with higher precedence execute first.

```
let result = 10 + 5 * 2;

console.log(result);
```

Output:

```
20
```

Because `*` has higher precedence than `+`.

Summary

Category	Purpose
Arithmetic	Mathematical calculations
Assignment	Assign or update values
Comparison	Compare values
Logical	Combine conditions
Increment/Decrement	Increase or decrease by 1
String	Join strings
Ternary	Short `if ... else`
Type	Check data type

Easy Way to Remember

- `+ - * / % **` → Arithmetic
- `= += -= *= /= %=` → Assignment
- `== === ≠ > < ≥ ≤` → Comparison
- `&& || !` → Logical
- `++ --` → Increment / Decrement
- `+` → String Concatenation
- `? :` → Ternary
- `typeof` → Type Checking

Interview Questions

Q1. What is the difference between `==` and `===`?

- `==` compares **only values** (type conversion occurs).
- `===` compares **both value and data type**.

```
console.log(5 == "5"); // true
console.log(5 === "5"); // false
```

Q2. What is the difference between `&&` and `||`?

- `&&` (AND): **All conditions must be true.**

- `||` (OR): **At least one condition must be true.**

```
console.log(true && false); // false
console.log(true || false); // true
```